Rapport – Application de gestion d'incidents

Authentification et autorisation avec Spring Security, JWT et React

Sécurité des systèmes distribués – Devoir d'application

Technologies : Spring Boot 3.3 · Spring Security · JWT (jjwt 0.13.0) · JPA · H2/MySQL · React 18 · Axios

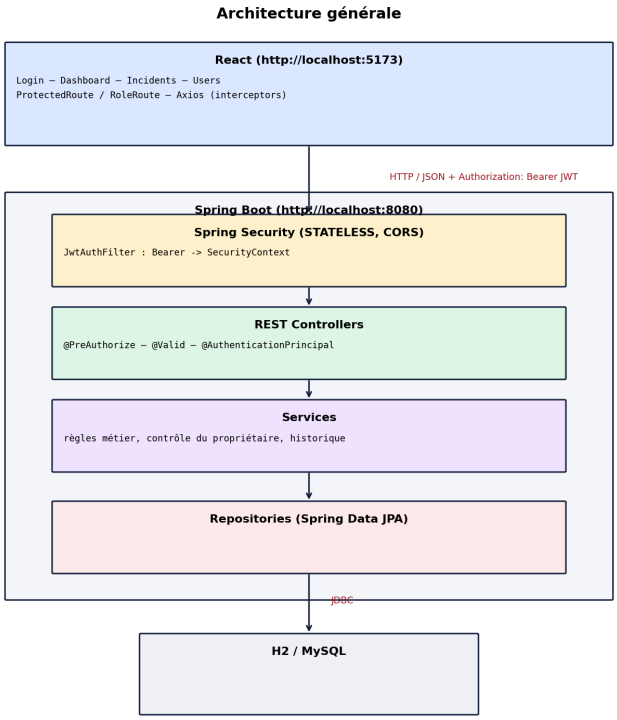
Étudiant : Ayoub Elhardi – ENSET Mohammedia, cycle ingénieur Cybersécurité et Confiance Numérique

1. Contexte et objectifs

L'application permet à des employés de déclarer des incidents informatiques ou de cybersécurité (malware, phishing, panne réseau, fuite de données…), à des techniciens de les traiter et à un administrateur de gérer les utilisateurs et d'affecter les incidents. Elle est composée d'un backend Spring Boot (API REST sécurisée par JWT) et d'un frontend React.

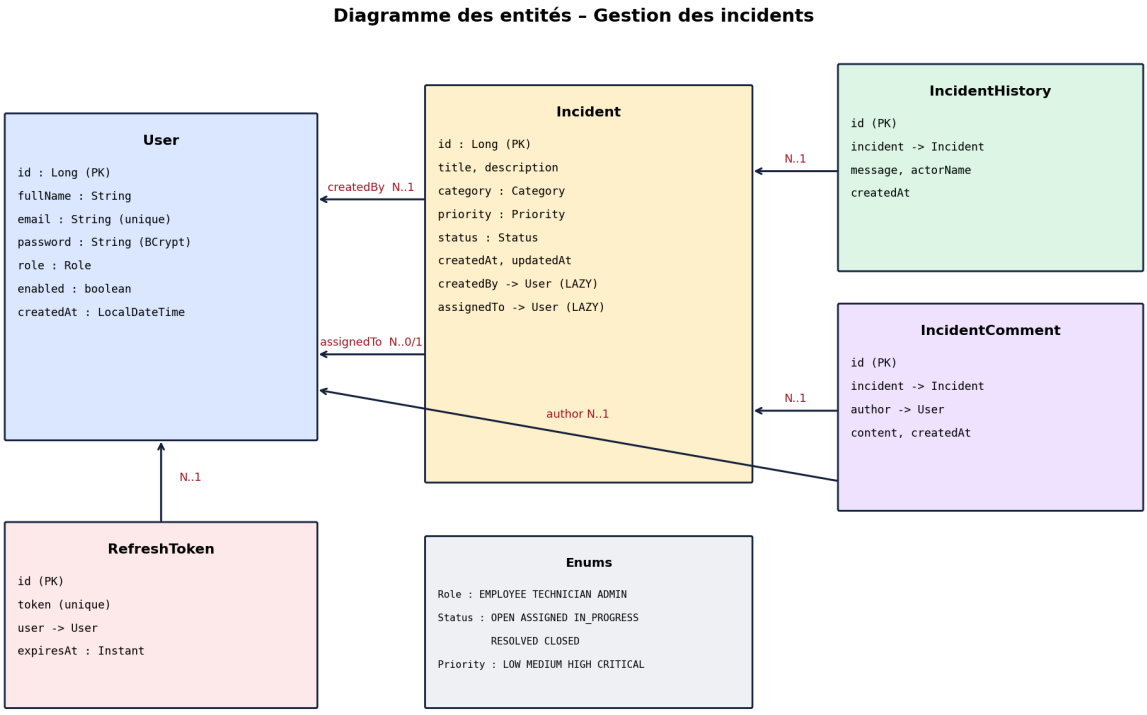
Rôle	Droits principaux
EMPLOYEE	Profil, déclarer un incident, consulter uniquement ses incidents
TECHNICIAN	Consulter les incidents affectés, changer leur statut, ajouter des notes
ADMIN	Tous les incidents et utilisateurs, activer/désactiver un compte, affecter un incident, statistiques

2. Architecture générale



Le frontend envoie des requêtes HTTP/JSON avec l'en-tête *Authorization: Bearer <JWT>*. Le filtre **JwtAuthFilter** valide le token, charge l'utilisateur et le place dans le *SecurityContext*. Les contrôleurs délèguent aux services (règles métier) qui utilisent les repositories JPA. Le backend est **stateless** : aucune session HTTP n'est conservée.

3. Modèle de données



Entités : **User**, **Incident**, **IncidentHistory**, **IncidentComment** et **RefreshToken** (bonus). Énumérations : Role, Category, Priority, Status.

3.1 Travail de recherche : relations JPA User ↔ Incident

@ManyToOne est placé du côté « plusieurs » : plusieurs incidents appartiennent à un utilisateur (*createdBy*) et plusieurs incidents peuvent être confiés à un même technicien (*assignedTo*, facultatif). La clé étrangère se trouve donc dans la table Incident, ce qui est le mapping le plus simple et le plus performant.

@OneToMany (côté User : liste des incidents) est possible mais je ne l'ai **pas** mis en place : une relation bidirectionnelle charge potentiellement une grande collection, peut provoquer des boucles de sérialisation JSON et n'est pas nécessaire puisque les requêtes passent par *IncidentRepository* (*findByCreatedBy…*, *findByAssignedTo…*). Ce choix limite aussi le risque de fuite de données : un User ne « contient » jamais les incidents d'autrui.

LAZY (chargement à la demande) est utilisé sur toutes les relations : on ne charge l'utilisateur lié que si on en a besoin, ce qui évite les jointures inutiles. **EAGER** (chargement immédiat) a été écarté car il provoque des jointures systématiques et le problème « N+1 » sur les listes. Comme *open-in-view* est désactivé, la conversion entité → DTO se fait dans des méthodes *@Transactional* des services, et les entités JPA ne sont jamais exposées en JSON.

4. Authentification

Endpoint	Description
POST /auth/register	Crée un compte avec le rôle EMPLOYEE imposé par le serveur. Le DTO d'inscription ne contient aucun champ « role » : un champ role envoyé est ignoré.
POST /auth/login	Vérifie le mot de passe (BCrypt). Retourne token, refreshToken, role, fullName, email. Compte désactivé → 403.
POST /auth/refresh	Échange un refresh token valide contre un nouveau couple access/refresh (rotation).

Endpoint	Description
POST /auth/logout	Révoque le refresh token côté serveur.

5. JWT

5.1 Travail de recherche

Claim : une information (paire clé/valeur) placée dans la charge utile (*payload*) du JWT. Les claims *enregistrés* sont standardisés (sub, iat, exp, iss…), les claims *personnalisés* sont libres (ici *role*). Le token est signé (HMAC-SHA256) : toute modification invalide la signature, mais le contenu n'est **pas chiffré** (il est seulement encodé en Base64URL) : on n'y met donc jamais de mot de passe.

```
Jwts.builder()
    .subject(user.getEmail())              // sub  : email
    .claim("role", user.getRole().name())  // claim personnalisé
    .issuedAt(now)                         // iat  : date de création
    .expiration(new Date(now + 300_000))   // exp  : 5 minutes
    .signWith(key).compact();

// Lecture d'un claim
Claims c = Jwts.parser().verifyWith(key).build().parseSignedClaims(token).getPayload();
String role = c.get("role", String.class);

// Vérification de l'expiration : parseSignedClaims lève ExpiredJwtException
// si exp est dépassé (c.getExpiration().before(new Date()) permet aussi de le tester)
```

Le filtre intercepte les exceptions de signature/expiration (*JwtException*) et n'authentifie pas la requête : l'*AuthenticationEntryPoint* renvoie alors 401.

6. Configuration Spring Security

• Routes publiques : **/auth/**** et **/h2-console/**** ; routes protégées : **/api/**** ; **/api/admin/**** réservé au rôle ADMIN.

• **SessionCreationPolicy.STATELESS**, CSRF désactivé (pas de cookie de session : le jeton est envoyé explicitement dans l'en-tête).

• **CORS** : seule l'origine http://localhost:5173 est autorisée (méthodes GET/POST/PUT/DELETE/OPTIONS, en-têtes Authorization et Content-Type).

• Le rôle et l'état « enabled » sont relus en base à chaque requête : un compte désactivé perd l'accès immédiatement, même avec un JWT encore valide.

• **AccessDeniedHandler** → 403 JSON ; **AuthenticationEntryPoint** → 401 JSON.

7. Gestion des rôles

Travail de recherche : hasRole, hasAuthority, @PreAuthorize, @EnableMethodSecurity

hasRole('ADMIN') vérifie l'autorité « ROLE_ADMIN » (Spring ajoute le préfixe ROLE_). **hasAuthority('ROLE_ADMIN')** compare l'autorité exacte, sans préfixe automatique. **@PreAuthorize** exprime la règle directement sur une méthode ou une classe ; elle n'est active que si **@EnableMethodSecurity** est présent sur une classe de configuration. J'utilise les deux niveaux : une règle globale sur /api/admin/** dans la chaîne de filtres, et @PreAuthorize sur chaque endpoint (défense en profondeur).

Endpoint	Règle
POST /api/incidents · GET /api/incidents/my	hasRole('EMPLOYEE')
GET /api/incidents/assigned	hasRole('TECHNICIAN')
GET /api/incidents · PUT …/assign/{id}	hasRole('ADMIN')
PUT …/status · POST …/comments	hasAnyRole('TECHNICIAN','ADMIN') + contrôle « incident affecté » pour un technicien
GET /api/incidents/{id}	tout utilisateur authentifié, puis contrôle propriétaire/technicien/admin dans le service
/api/admin/**	hasRole('ADMIN') (chaîne de filtres + @PreAuthorize)

8. Contrôle du propriétaire d'un incident

Un employé peut appeler *GET /api/incidents/12* directement avec Postman. Le backend ne fait donc jamais confiance au frontend : l'utilisateur courant est obtenu depuis le **SecurityContext** (injecté par *@AuthenticationPrincipal*, alimenté par le JWT), jamais depuis un paramètre de la requête, puis comparé au propriétaire de l'incident.

```
case EMPLOYEE -> {
    if (!incident.getCreatedBy().getId().equals(current.getId()))
        throw new ForbiddenException("Accès interdit : cet incident ne vous appartient pas");
}
case TECHNICIAN -> { /* doit être le technicien affecté */ }
case ADMIN      -> { /* accès total */ }
```

Authentication représente l'identité authentifiée ; *SecurityContextHolder* la stocke pour la durée de la requête ; *Principal* est l'objet utilisateur (ici l'entité User, qui implémente UserDetails).

9. Règles métier

#	Règle	Mise en œuvre
1-2	Nouvel utilisateur = EMPLOYEE, jamais ADMIN	AuthService.register fixe Role.EMPLOYEE ; aucun champ role dans RegisterRequest
3	Mots de passe BCrypt	BCryptPasswordEncoder
4	Employé : uniquement ses incidents	IncidentService.checkCanRead + /my filtré par utilisateur
5	Technicien : uniquement les incidents affectés	isAssignedTo() dans updateStatus et addComment
6	Seul l'admin affecte	@PreAuthorize('ADMIN') sur /assign
7	Compte désactivé ne se connecte plus	ForbiddenException au login ; filtre JWT refuse le compte ; refresh tokens révoqués
8	Incident CLOSED non modifiable	BadRequestException (sauf réouverture par un ADMIN)

10. Validation et gestion des exceptions

10.1 Validation (recherche : @NotBlank, @Email, @Size, @Valid)

Les DTO (records Java) portent les contraintes Bean Validation : **@NotBlank** (non nul et non vide après trim), **@Email** (format), **@Size(min, max)** (longueur), **@NotNull** (pour les enums). L'annotation **@Valid** sur le paramètre *@RequestBody* du contrôleur déclenche la validation ; en cas d'échec Spring lève *MethodArgumentNotValidException*, traduite en 400 avec la liste des champs fautifs.

10.2 @RestControllerAdvice / @ExceptionHandler

Une classe annotée **@RestControllerAdvice** (version REST de @ControllerAdvice) centralise le traitement des exceptions ; chaque méthode **@ExceptionHandler** associe un type d'exception à une réponse JSON uniforme *{ "status": 404, "message": "Incident introuvable" }*.

Situation	Exception	Code
Utilisateur / incident / technicien introuvable	NotFoundException	404
Email déjà utilisé	ConflictException	409
Mauvais mot de passe (login)	UnauthorizedException	401
Accès à une ressource d'autrui, rôle insuffisant	ForbiddenException / AccessDeniedException	403
Données invalides, règle métier violée	MethodArgumentNotValidException / BadRequestException	400

10.3 Différence entre 400, 401, 403 et 404

• **400 Bad Request** : la requête est mal formée ou invalide (validation, règle métier). Le client doit corriger sa demande.

• **401 Unauthorized** : l'identité n'est pas établie (pas de JWT, JWT invalide ou expiré, mauvais identifiants). Le client doit s'authentifier.

• **403 Forbidden** : l'identité est connue mais n'a pas le droit (rôle insuffisant, incident d'un autre). S'authentifier de nouveau ne change rien.

• **404 Not Found** : la ressource n'existe pas.

11. Frontend React

• **Structure** conforme au sujet : components (Navbar, ProtectedRoute, RoleRoute), pages (Login, Register, Dashboard, Profile, MyIncidents, NewIncident, IncidentDetails, AssignedIncidents, AdminUsers, AdminIncidents), services (api, authService, incidentService).

• **JWT** stocké dans le localStorage après connexion. Un **interceptor de requête Axios** ajoute automatiquement « Authorization: Bearer <JWT> ».

• **Expiration (5 min)** : un **interceptor de réponse** (axios.interceptors.response.use) intercepte le 401, supprime le JWT, redirige vers /login et affiche « Votre session a expiré ». Les appels /auth/* sont exclus (un mauvais mot de passe renvoie aussi 401 et ne doit pas déclencher ce comportement).

• **ProtectedRoute** bloque l'accès sans authentification ; **RoleRoute** contrôle le rôle (/admin/users réservé à ADMIN). Ces protections améliorent l'expérience utilisateur ; la sécurité réelle reste côté backend.

• **Dashboards** adaptés au rôle (EMPLOYEE, TECHNICIAN, ADMIN) alimentés par GET /api/dashboard.

12. Bonus : Refresh Token

Access token : JWT à courte durée de vie (5 min ici), envoyé à chaque requête ; s'il est volé, la fenêtre d'exploitation est courte. **Refresh token** : jeton opaque (UUID) à longue durée de vie (7 jours), stocké en base, utilisé uniquement pour obtenir un nouvel access token via POST /auth/refresh.

Renouvellement : le client reçoit un 401, appelle /auth/refresh avec son refresh token, reçoit un nouveau couple (access, refresh) — l'ancien refresh token est supprimé (rotation) — puis rejoue la requête initiale. Si le refresh échoue, l'utilisateur est redirigé vers /login. **Déconnexion** : POST /auth/logout supprime le refresh token côté serveur, puis le navigateur efface ses jetons ; le refresh token ne peut plus être réutilisé. Un compte désactivé voit ses refresh tokens révoqués.

13. Scénario de test (section 24 du sujet)

Étape	Résultat attendu
Créer 2 EMPLOYEE (inscription), TECHNICIAN et ADMIN (comptes de démonstration / création par l'admin)	201 ; rôle EMPLOYEE pour les inscriptions
Employé 1 déclare 2 incidents, employé 2 en déclare 1	201 ; statut OPEN ; historique « Incident créé »
Employé 1 consulte l'incident de l'employé 2	403 Accès interdit
Admin liste tous les incidents, affecte un incident au technicien	200 ; statut ASSIGNED ; historique « Affecté à … »
Technicien : liste ses incidents, IN_PROGRESS puis RESOLVED, ajoute une note	200 / 201 ; historique complété
Employé 1 revoit son incident	Statut RESOLVED, historique et note visibles

14. Tests de sécurité obligatoires (section 25)

#	Test	Résultat attendu
1	API protégée sans JWT	401
2	JWT invalide	401
3	JWT expiré	401
4	EMPLOYEE sur /api/admin/users	403
5	EMPLOYEE sur l'incident d'un autre	403
6	TECHNICIAN modifie un incident non affecté	403
7	Utilisateur désactivé : connexion	403 « Compte désactivé »
8	Inscription avec email existant	409
9	Création d'incident avec données invalides	400 + détail par champ

Ces tests sont fournis sous forme de collection Postman (tests/incident-security.postman_collection.json) et de script automatisé (tests/security_tests.py). Les captures d'écran Postman et de l'application sont à insérer ci-dessous après exécution.

Captures Postman

[À insérer : une capture par test de 1 à 9]

Captures de l'application

[À insérer : Login, Dashboard (3 rôles), liste d'incidents, détail avec historique, page d'administration, message « session expirée »]

15. Limites et améliorations possibles

• Le JWT est stocké dans le localStorage (simple, exigé par le sujet) mais vulnérable à un vol par XSS : un cookie HttpOnly + SameSite serait plus sûr.

• La clé de signature doit provenir d'une variable d'environnement ou d'un coffre de secrets en production (JWT_SECRET).

• Ajouter une limitation des tentatives de connexion (anti brute-force), HTTPS obligatoire et des tests unitaires/intégration automatisés.

• Un JWT déjà émis reste valide jusqu'à son expiration ; la relecture du compte en base à chaque requête compense pour la désactivation.